

JavaScript

Basics, Praxis und Neuerungen

In eigener Sache

- Zweifelsfrei werden zur Entwicklung von Web-Seiten heutzutage Frameworks wie Angular, React oder Vue.js verwendet
- In dieser Veranstaltung konzentrieren wir uns auf die darunter liegenden Grundlagen:
 - **Grundlagenverständnis durch JavaScript statt „Framework-Magie“:** DOM, Events, Scope, this, Prototypen, Module, **asynchrones Verhalten (Promises/async-await)** sind die Basis – Frameworks bauen darauf auf
 - **Debugging:** Wer weiß, was der Browser wirklich macht kann Fehler in Frameworks gezielter finden und lösen. In der Dev-Konsole sieht man JavaScript
 - **Framework-Agnostik:** Bessere Lernkurve bei der Verwendung oder Wechsel eines Frameworks
 - **Web-APIs als Kernkompetenz:** Konzepte sind unabhängig von Frameworks und überall nutzbar
 - **Stufenweises lernen:** Frameworks können überfrachtet wirken und eine hohe kognitive Last erzeugen (Angular)

Themenübersicht

Basics

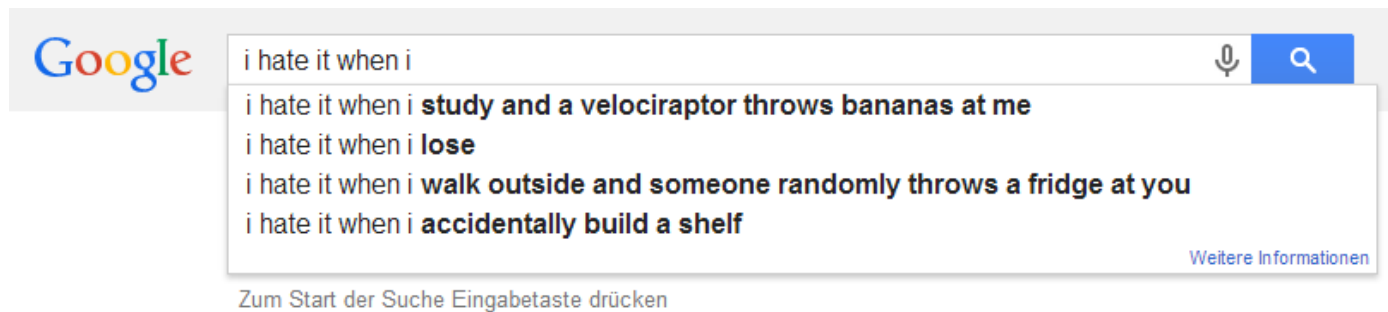
- Syntax
- Funktionen & Closures
- Klassen (ein wenig)
- **Event Loop**

JavaScript im Einsatz

- Node.js
- DOM
- Promises
- AJAX
- Events
- Typescript

Motivation

JavaScript ist überall (im Browser)



- Beispiele
 - > Google-Suche
 - > Facebook-Timeline
 - > X
 - > Browser-Plugins
 - > ...



Motivation

(Ehemals) schlechter Ruf der Sprache

- Die Sprache wurde in nur 10 Tagen entwickelt, um schnell eine Skriptsprache für den Netscape Navigator bereitzustellen
- Hastige und fragliche Desingentscheidungen
- 1997 wurde der Sprachkern von JavaScript durch die Organisation ECMA International standardisiert und als ECMAScript veröffentlicht

Node.js in 2009 (serverseitiges JavaScript)

- Event-basierter Ansatz eine Stärke der Sprache



Heutzutage

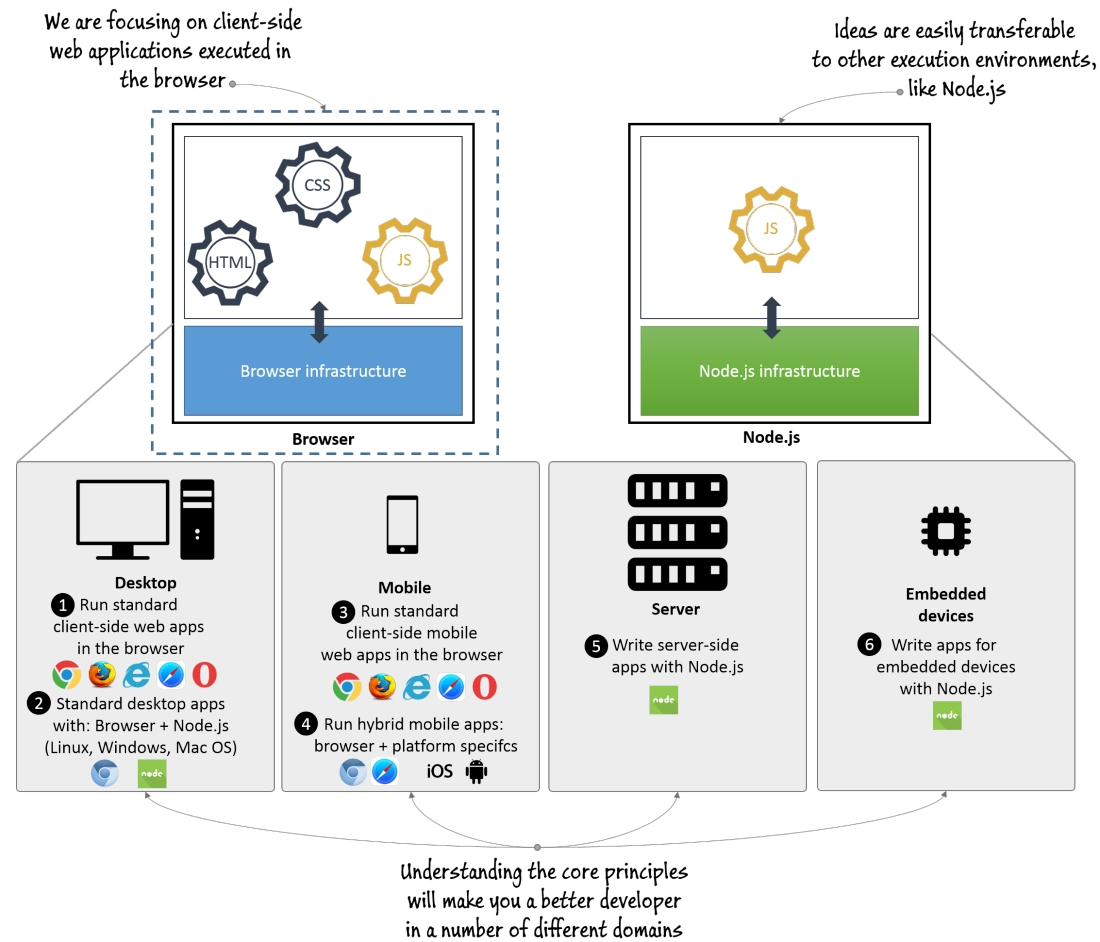
- Unterstützung von nahezu jedem Gerät (mit Browser)



APACHE
CORDOVA™

Motivation

<https://dzone.com/articles/secrets-javascript-ninja>



HTML & CSS bereits kennengelernt

- Trennung (Separation of Concerns) ist wichtig

Kette Inhalt, Präsentation und Logik

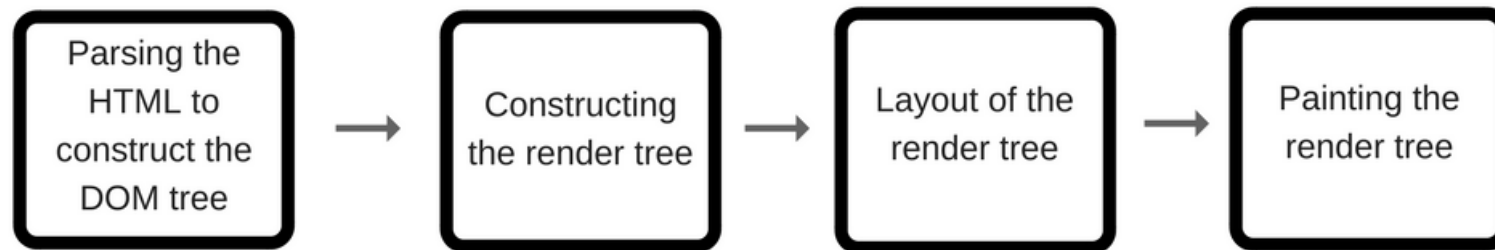
- Semantisches Markup bzw. strukturelle Informationen in HTML
- Bezieht sich auf die (dynamischen) Inhalte (z.B. „dieser Text ist wichtig“)
- Informationen zur Darstellung nur im Stylesheet
- JavaScript wird Logik liefern

Vorteile

- Die aus HTML und CSS abgeleitete Präsentation ist durch JavaScript dynamisch anpassbar, ohne neuen HTML-Code laden zu müssen
- Je „sauberer“ die Trennung von HTML/CSS desto einfacher das Zusammenspiel mit JavaScript

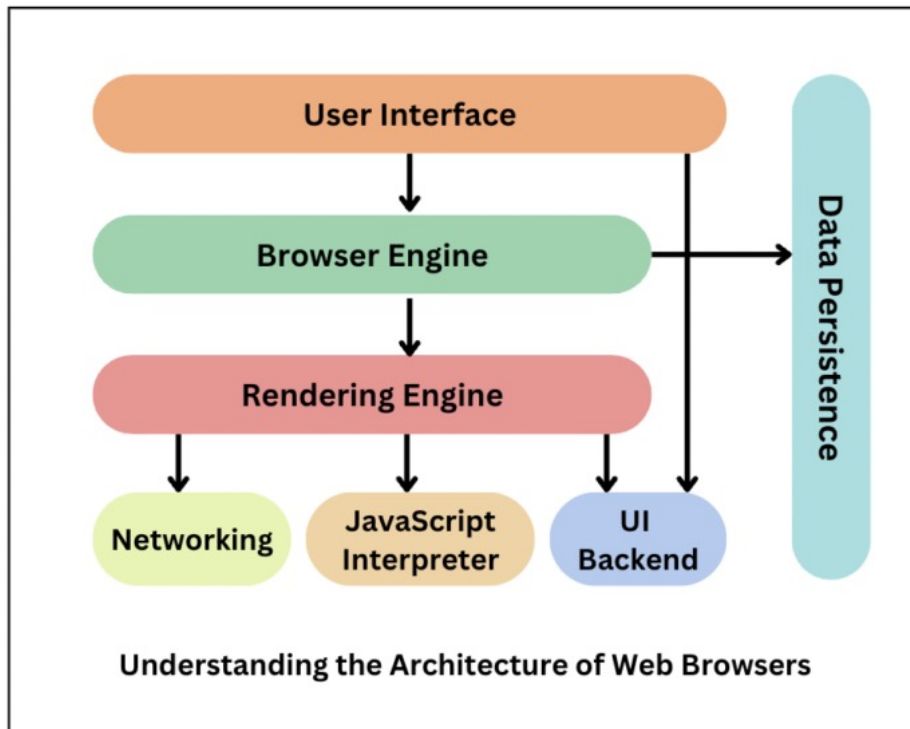
Im Browser

<https://blog.sessionstack.com/how-javascript-works-the-rendering-engine-and-tips-to-optimize-its-performance-7b95553baeda>



Im Browser

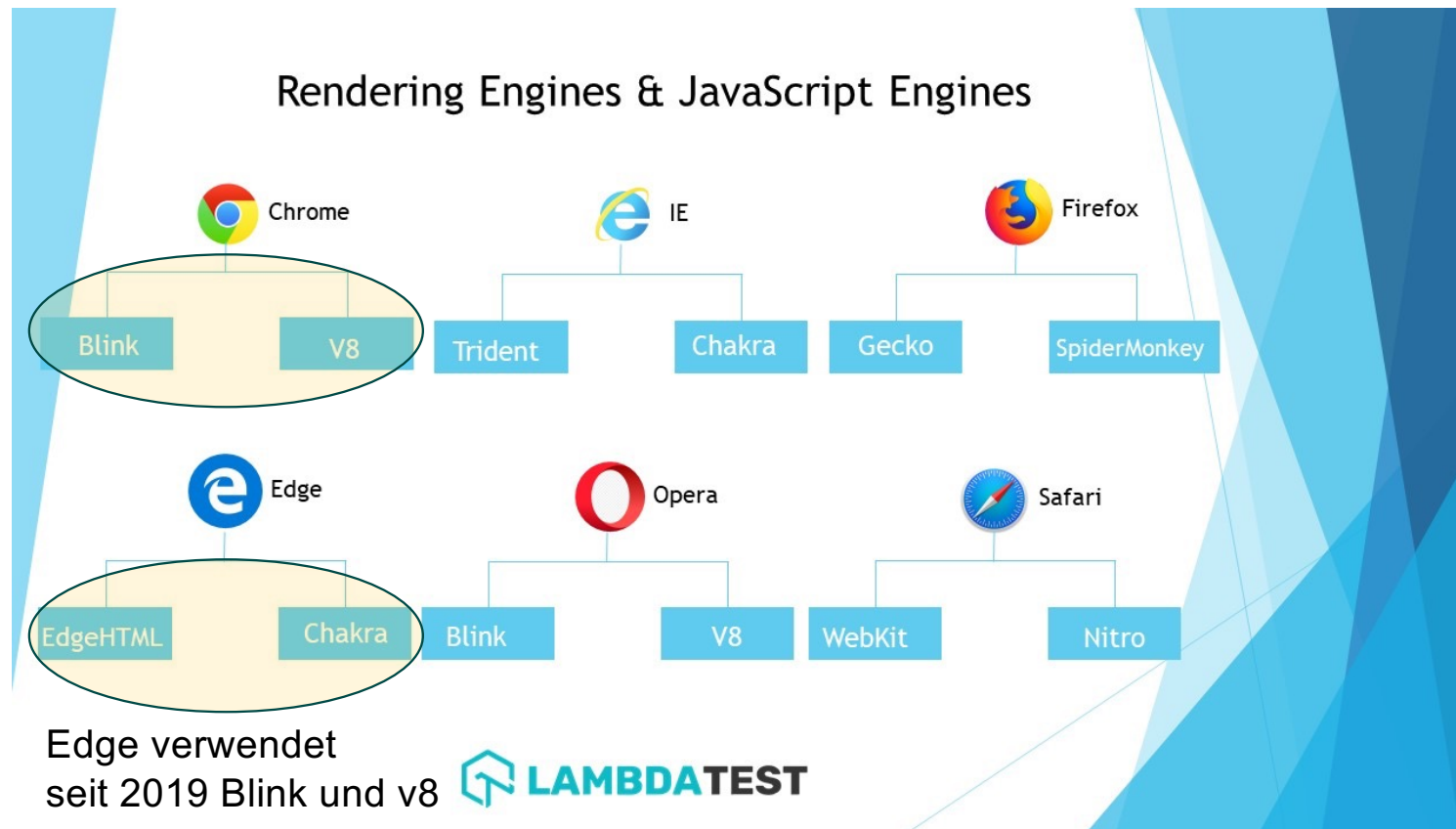
<https://www.browserstack.com/guide/browser-rendering-engine>



- Browser Engine ist für die Koordination zwischen verschiedenen Komponenten des Browsers zuständig
- Rendering Engine ist speziell für die visuelle Darstellung von Webseiten verantwortlich
- JavaScript-Interpreter wird von der Rendering Engine angesprochen, wenn dynamische Änderungen am DOM oder CSS vorgenommen werden müssen.
- Diese Zusammenarbeit ermöglicht interaktive und dynamische Webseiten

Rendering und JavaScript-Engines

<https://www.lambdatest.com/blog/browser-engines-the-crux-of-cross-browser-compatibility/>



Die Besonderheiten beim Ablauf von JavaScript im Browser werden wir zu einem späteren Zeitpunkt noch behandeln

Rendering und JavaScript-Engines

<https://medium.com/@valerii.sukhov/uth-browsers-rendering-engines-9d9731cc8d08>

BROWSER ENGINES



**BLINK
ENGINE**



**WEBKIT
ENGINE**



**TRIDENT
ENGINE**



**GECKO
ENGINE**

Microsoft Edge nun auch Chromium-basiert (Legacy Edge ist ausgelaufen)

Grundlagen JavaScript

Entwickelt 1995 von Brendan Eich

ECMAScript (ES) liefert den Ursprung und regelt in den unterschiedlichen Versionen den Standard und damit den Funktionsumfang und die Syntax

- Hatte ursprünglich auch andere Sprachen ActionScript und Jscript
- Wichtiger Schritt: **ECMAScript 6** (ES6) in 2015

Objektorientiert, aber klassenlos (Klassen werden aber ab ES6 nachempfunden)

- OOP mittels Prototyping

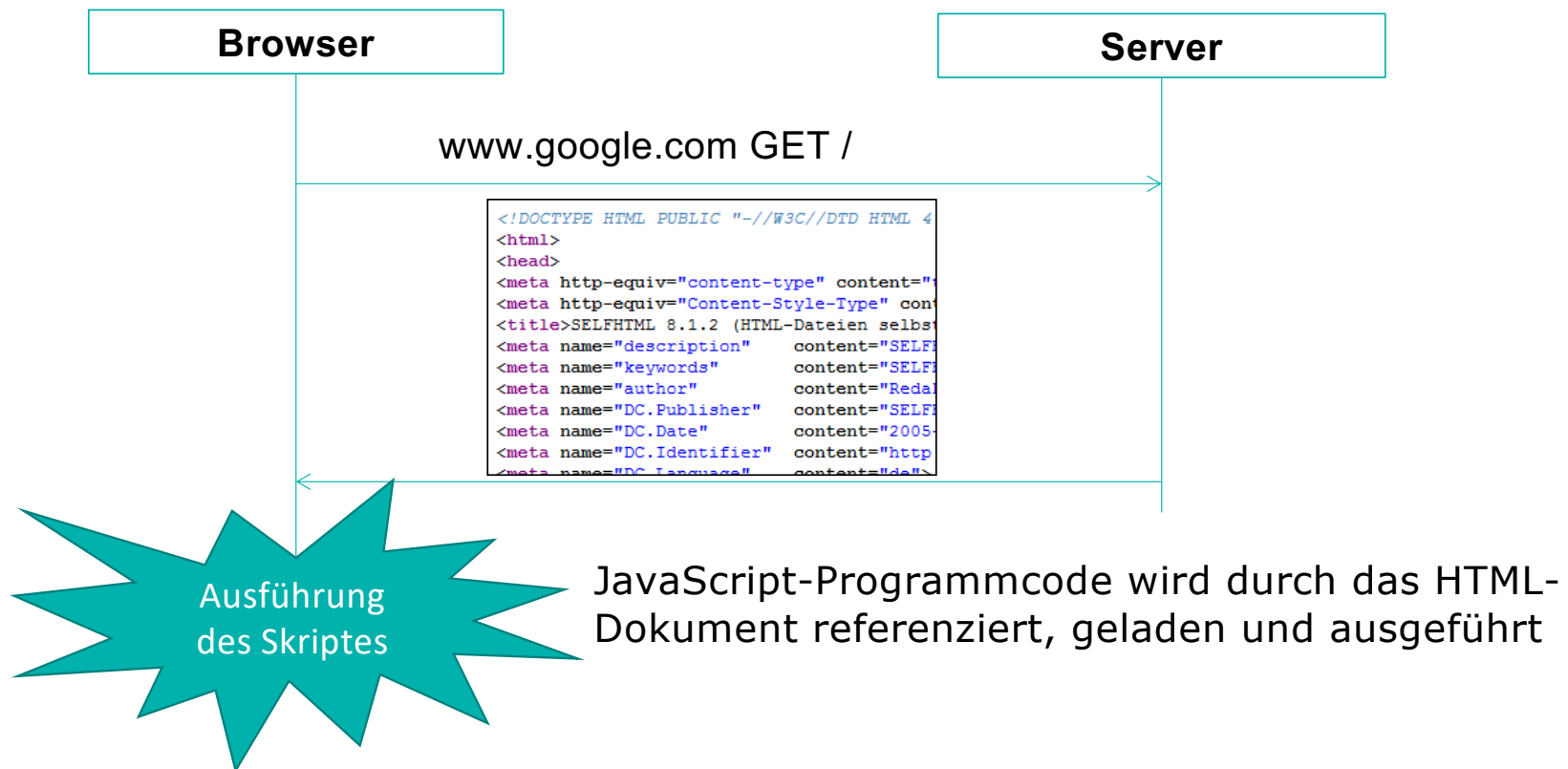
Merkmale der Sprache

„It's a **single-threaded non-blocking**
asynchronous event-based language!“

Das Thema single-threaded wird uns noch beschäftigen.

Non-Blocking und Asynchronous werden eine Schlussfolgerung sein

Ausführung des Skriptes auf dem Rechner des „Websurfers“



Grundlagen

Clientseitige Ausführung des Skriptes (für jetzt)

- Ursprünglich eigentlich eine clientseitige Programmiersprache
- Code ist eingebettet in HTML-Dokumente (besser über einen Verweis)
- Ausführung des Codes geschieht zwar automatisch, dieser wird aber im Wesentlichen über Events gesteuert
- Strikte Trennung von JavaScript-Code, CSS und HTML empfehlenswert

Zugriff auf das Browserfenster und dem darin enthaltenen Dokument (DOM)

- Events erlauben die Möglichkeit auf Benutzereingaben zu reagieren
- Manipulation des DOM-Baums somit möglich, ohne ein neues HTML zu laden (Single Page Application)
- Auch ist eine Kommunikation mit dem Quell-Server möglich

Auf (so gut wie) jeder Plattform bzw. jedem Browser unterstützt

Wir beschäftigen uns auch mit der serverseitigen Ausführung!

Hinweis:

Wir verwenden zunächst die JavaScript-Engine im Browser. Dies hat einige Konsequenzen

- Der Code muss in eine Web-Seite eingebettet werden
- Ausgaben auf der Konsole sind nur über die Konsole im Entwicklermodus sichtbar
- Der Code wird zu bestimmten Zeitpunkten geladen und gestartet (kommt noch)
- Das editieren des Codes ist über Debugging im Entwicklermodus des Browsers möglich

Grundlagen

Vorteile von JavaScript im Browser

- Veränderung der Webseite (DOM-Modell) ohne diese neu laden zu müssen
 - > Reduziert Netznutzung und Ladezeit – Wir laden nicht mehr ganze HTML-Dokumente sondern nur noch die Information, die wir benötigen
 - > Rendering wird schneller
 - > Ermöglicht so eine schnellere Interaktion mit dem Benutzer (Vorschlagsliste)
 - > Auslagern von Funktionen und Logik auf den Client möglich (Rich Internet Client)
 - > Steigert die Nutzbarkeit und Benutzerfreundlichkeit durch verbesserte Steuerung der durch den Benutzer verursachten Ereignisse
- Typische Anwendungsfälle:
 - > Interaktion mit dem Benutzer
 - Validierung von Formulareingaben vor der Übertragung zum Server
 - Anzeige von Dialogfenstern
 - Vorschlagen von Suchbegriffen während der Eingabe
 - Reaktion auf Maus-Operationen
 - > Entlastung des Servers und Nutzung von Rechenkapazitäten des Clients
 - > Single-Page-Anwendung – Nachladen relevanter Information und Integration dieser in den DOM-Baum

Beispiel 1: (Einbindung von Code)

```
<!DOCTYPE html>
<html>
<head>
  <title>Skriptprogrammierung</title>
  <link rel="stylesheet" href="style.css" />
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Willkommen im Kurs!</h1>
  <p>...</p>
</body>
</html>
```

Defer lädt die Daten parallel und führt dazu, dass der JavaScript-Code erst dann startet, wenn der DOM-Baum vollständig erstellt wurde

Syntax

Eventgetrieben durch den Browser

Beispiel 1: script.js

```
window.onload = function () {  
    alert('Willkommen!');  
};
```

Ist das Dokument vollständig mit allen abhängigen Ressourcen geladen, so geben wir eine Nachricht aus

Hinweise:

- Klassische Ausgabe gibt es nicht (JavaScript-Konsole: console.log)
- **DOMContentLoaded** wird schon früher angesprochen und wird im Sinne einer schnelleren Benutzerinteraktion empfohlen
- Die Nutzung des eben genannten defer-Attributs hat sich mittlerweile etabliert und sollte als Standard genutzt werden
- **Verwenden Sie nicht window.onload am Anfang eines JavaScript-Programmes, das ist ein Anti-Pattern**

Syntax

Eventgetrieben durch den Browser

Alternative:

```
document.addEventListener("DOMContentLoaded", function() {  
    alert('Willkommen!');  
});
```

Hinweis: Nutzen Sie defer!

Beispiel 2:

```
<!DOCTYPE html>
<html>
<head>
  <title>Skriptprogrammierung</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <button onclick="doStuff();" >Klick mich!</button>
  <script>alert('Willkommen!');</script>
</body>
</html>
```

Problem: Keine Trennung von HTML und JavaScript!

- JavaScript besser in eigene Dateien auslagern

JavaScript

Einbinden über den Script-Tag

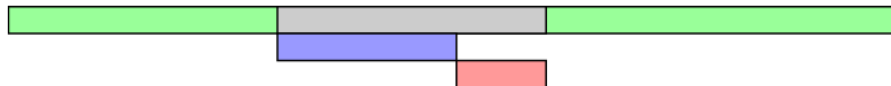
<https://www.growingwiththeweb.com/2014/02/async-vs-defer-attributes.html>

Legend

- HTML parsing
- HTML parsing paused
- Script download
- Script execution

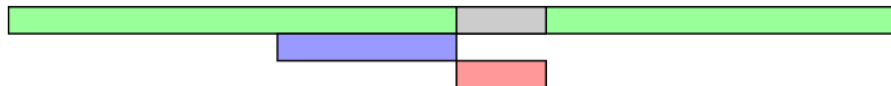
<script>

Let's start by defining what `<script>` without any attributes does. The HTML file will be parsed until the script file is hit, at that point parsing will stop and a request will be made to fetch the file (if it's external). The script will then be executed before parsing is resumed.



<script async>

`async` downloads the file during HTML parsing and will pause the HTML parser to execute it when it has finished downloading.



<script defer>

`defer` downloads the file during HTML parsing and will only execute it after the parser has completed. `defer` scripts are also guaranteed to execute in the order that they appear in the document.



Nur geringe Ähnlichkeiten mit Java

Bekannt aus PHP:

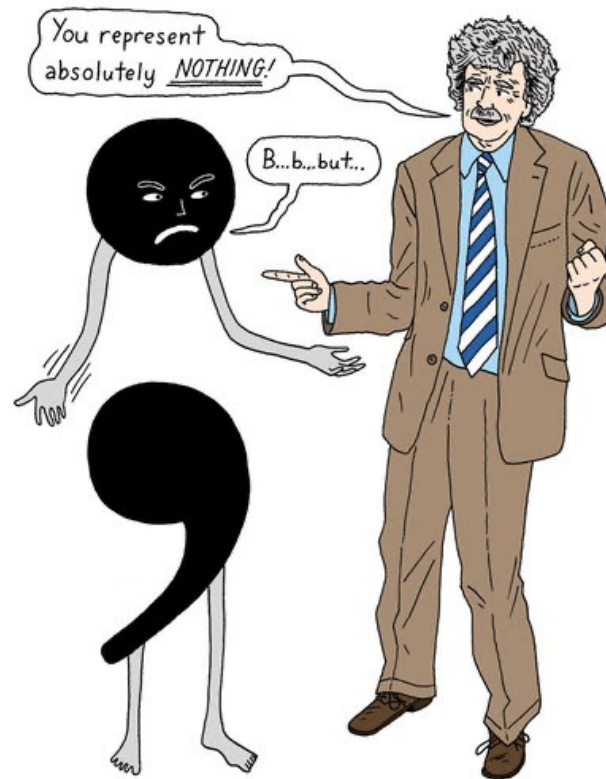
- Variablen ohne festen Typen (TypeScript)
- Funktionen ohne festen Rückgabetypen
- Keine Überladung von Funktionen

Viele Konzepte vollkommen neu

- Besonderheiten, die man aus anderen Sprachen nicht kennt (Prototyping, Gültigkeitsbereich)
- Semikolons sind optional

Syntax

<https://opinionator.blogs.nytimes.com/2012/07/02/semicolons-a-love-story/>



Der Code kann mit Semicolons
aber lesbarer werden

Syntax

Variablen – Der ursprüngliche Weg – Nicht mehr verwenden

Beispiel:

```
var wert; // Instanziierung ohne Wert
wert = 5; // Variable wird auf 5 gesetzt
var timId = 'xy123456'; // Wert sofort setzen
```

Variablen-Deklaration mittels Schlüsselwort „var“ (alte Syntax)

- Kein fester Typ!
- Nicht initialisierte Variablen haben den Wert *undefined*

Variablen, die außerhalb von Funktionen deklariert werden sind global! Innerhalb von Funktionen sind neu deklarierte Variablen lokal

Syntax

Variablen – Der ursprüngliche Weg

Variablen-Deklaration mittels des Schlüsselworts „**var**“ **ignorieren Blöcke, beachten aber Funktionen (Function-Scope)!**

```
if (true) {  
  // "if" block scope  
  var count = 0;  
  console.log(count); // 0  
}  
console.log(count); // 0
```

```
function run() {  
  // "run" function scope  
  var message = 'Run, Forrest, Run!';  
  console.log(message); // 'Run, Forrest, Run!'  
}  
  
run();  
console.log(message); // throws ReferenceError
```

<https://dmitripavlutin.com/javascript-scope/>

Syntax

Variablen – Der neue Weg (ES6, 2015) – Beseitigt viele Probleme

Variablen-Deklaration mittels des Schlüsselwortes „let“ oder Konstante mittels „const“ beachten Blöcke (Block-Scope)

```
if (true) {  
  const message = 'Hello';  
}  
console.log(message); // ReferenceError: message is not defined
```

```
while (/* condition */) {  
  // "while" block scope  
  const message = 'Hi';  
  console.log(message); // 'Hi'  
}  
console.log(message); // => throws ReferenceError
```

```
{  
  // block scope  
  const message = 'Hello';  
  console.log(message); // 'Hello'  
}  
console.log(message); // throws ReferenceError
```

<https://dmitripavlutin.com/javascript-scope/>

Syntax

Variablen – Der neue Weg (ES6, 2015)

Wann immer eine Variable außerhalb von Funktionen deklariert wird (let, var, const), so wird sie global

```
let carName = "Volvo";  
// code here can use carName  
  
function myFunction() {  
  // code here can also use carName  
}
```

https://www.w3schools.com/js/js_scope.asp

JavaScript has 8 Datatypes

1. String
2. Number
3. BigInt
4. Boolean
5. Undefined
6. Null
7. Symbol
8. Object

The Object Datatype

The object data type can contain:

1. An object
2. An array
3. A date

ECMAScript 6 liefert unigenen Wert, der unveränderlich ist

```
let id1 = Symbol("id");  
let id2 = Symbol("id");  
  
alert(id1 == id2); // false
```

Weitere Informationen: https://www.w3schools.com/js/js_datatypes.asp

Typumwandlung

- Implizit – Beispiel:

```
let num = '3' + 5; // 35
let b1  = !! (0);    // false
let b2  = !! ('0');  // true
```

- Explizit – Beispiele:

```
let foo = parseInt('5.3'); // 5
let bar = parseFloat('5.3'); // 5.3
let str = (5.3).toString(); // "5.3"

Number(value: " 1 ") //1
Number(value: "") //0

String(value: true) //"true"
true.toString() //"true"
String(value: false) //"false"
false.toString() //"false"
```

Vergleich von Variablen

- „**falsy values**“ (diese Werte liefern bei Interpretation als Boolean false)

```
Boolean( value: false) //false
Boolean( value: 0) //false
Boolean(NaN) //false
Boolean( value: "") //false
Boolean( value: null) //false
Boolean( value: undefined) //false
```

- **Alle anderen Werte** (z.B. auch leere Arrays) ergeben **true**
> „truthy values“

Weitere Informationen: <https://developer.mozilla.org/en-US/docs/Glossary/Falsy>

Vergleich von Variablen

```
let a = (false == ""); // true
let b = (false == 0);   // true
let c = (0 == "");      // true

let d = (null == null); // true
let e = (null == false); // false
let f = (undefined == undefined); // true
let g = (undefined == null); // true

let h = (NaN == null); // false
let i = (NaN == NaN);  // false !!!
```

Auch JavaScript bietet den **typstarken Vergleich: a === b**

Verkettung mittels +-Operator

Literale:

```
'Ich bin ein String'
```

```
"Ich auch!"
```

- Kein Unterschied (anders als bei PHP), trotzdem ist Einheitlichkeit sinnvoll

Steuerzeichen können mittels Backslash angegeben werden

- Beispiel: `'Zeilenumbruch folgt\n2. Zeile!'`

Benutzung des Anführungszeichens muss maskiert werden

```
document.write("Hallo, Sie befinden sich auf \"meiner Seite\");
```

Verkettung über +

Syntax

Template Literals – ja wir verwenden den schrägen `

Syntaktischer „Sugar“ um die Ersetzung von Variablen in Zeichenketten lesbar zu machen

` ... \${ Ausdruck mit Variablen } ... `

```
let a = 5;
let b = 10;
console.log('Fifteen is ' + (a + b) + ' and\nnot ' + (2 * a + b) + '.');
// "Fifteen is 15 and
// not 20."
```

```
let a = 5;
let b = 10;
console.log(`Fifteen is ${a + b} and
not ${2 * a + b}.`);
// "Fifteen is 15 and
// not 20."
```

Eigenschaften

- **Keine feste Größe**
- Als Wert ist alles erlaubt
 - > Arrays in Arrays sind möglich
 - > Objects in Arrays sind möglich
- Arrays können keine Lücken haben (**Lücken werden mit undefined aufgefüllt**)

Assoziative Arrays

- **Gibt es nicht!**
 - > Dafür gibt es Objekte (die sich aber anders verhalten – ein Array hat beispielsweise eine length-Methode)
 - > Arrays sollten besser verwendet werden, wenn wir die Ordnung der Indizes sinnvoll verwenden können

Weitere Informationen: https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Global_Objects/Array

Beispiele

- Erstellung eines Arrays

```
var emptyArr1 = new Array(); // leeres Array erstellen
var emptyArr2 = [];          // Alternative (präferiert)

var arr = ['one', 2];
```

- Zugriff auf Werte

```
var one = arr[0];           // one
var two = arr[arr.length-1]; // 2

arr[4] = 4;
var len = arr.length;       // 5
```

Konstanten sollten mit const angelegt werden, Sichtbarkeit wie let!

```
const dieLoesungallerFragen = 42;  
dieLoesungallerFragen = 13; // ergibt einen Fehler
```

```
var dieLoesungallerFragen = 13 // ergibt einen Fehler  
let dieLoesungallerFragen = 13 // ergibt einen Fehler  
const wertNotwendig; // ergibt einen Fehler
```

Vorsicht:

const-Variablen sind **durchaus änderbar** und zwar wenn es sich um ein Objekt/Array handelt, dann können die Attribute geändert werden, nicht aber die Objektreferenz

Grundsätzliches bekannt aus Java (jetzt auch aus PHP!)

- if
- else if
- else

- for
- while
- do-while
- break
- continue

for-in-Schleife

```
for (let key in obj) {  
    alert(key + ': ' + obj[key]);  
}
```

Durchläuft das Objekt (oder Array) und gibt die enthaltenen Eigenschaften aus

- Besser mit Bedacht zu verwenden
 - > Arrays möglichst mit `forEach` oder `for ... of arrayvar` verwenden
 - > Besser `Object.keys(obj)` verwenden
 - > Vorsicht bei Objekten (besonders bei Änderungen der Prototypen im Durchlauf) - mehr dazu später

Weitere Informationen: <https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Statements/for...in>

for-of-Schleife

```
const array1 = ['a', 'b', 'c'];  
  
for (const element of array1) {  
    console.log(element);  
}
```

Tatsächlich ändert sich element nicht in einem Durchlauf

```
// expected output:  
// a  
// b  
// c  
// array1.forEach(FUNKTION) ...
```

Iteriert über die Werte eines entsprechenden Iterable-Objekts
For-in iteriert über die aufzählbaren Eigenschaften eines Objekts

Weitere Informationen: <https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Statements/for...of>

Syntax

Komplexe Datentypen und Funktionen

- Basis für komplexe Datentypen sind Objekte
- Objekte dienen dazu, Eigenschaften und Methoden zu bündeln – ähnlich wie man es aus der Klassenprogrammierung kennt (intern aber anders)
- JavaScript bringt zahlreiche **eingebaute Objekttypen** mit, z. B. `Array`, `Date`, `Map` oder `Math`. In Browserumgebungen kommen zusätzlich **Web-APIs** hinzu, z. B. `document`, `window` oder `location`
- Darüber hinaus können **eigene Objekte** definiert werden
- Objekte in JavaScript unterscheiden sich intern von anderen Programmiersprachen
- Tatsächlich sind auch Funktionen Objekte (First Class Objects)
- In JavaScript gibt es den Basistyp **Object**. Er bildet die Grundlage der prototypbasierten Vererbung: Über die **Prototypenkette** erben Objekte Eigenschaften und Methoden (Details später). Alle Objekte stammen von diesem obersten Kernobjekt ab

Syntax

Objekte & Funktionen (native Erklärung)

Objekte und Funktionen unterscheiden sich stark von vielen anderen Programmiersprachen

- Typen und Vererbung werden in den meisten Programmiersprachen durch Klassen gelöst
 - JavaScript besitzt keine (echten) Klassen!*&ul> - > Stattdessen gibt es **Konstruktorfunktionen** und **Prototypen** für Objekte
 - > **Objekte kapseln** aber auch hier **Eigenschaften und Methoden**
 - > Jedes Objekt hat eine prototype-Eigenschaft, die für Vererbung genutzt werden können (müssen)
- **Funktionen sind tatsächlich First-Class-Objects:**
 - > Jede Funktion ist eigentlich ein JavaScript-Objekt
 - > Funktionen können somit Attribute und weitere Methoden haben
- **Eine Methode ist eine Objekt-Eigenschaft mit einem Funktionswert** (also eine Referenz auf eine Funktion)

*Seit ES6 gibt es das Schlüsselwort „class“, aber die hiermit erstellen Klassen verhalten sich anders als traditionelle Klassen aus anderen Sprachen, es bleibt prinzipiell beim Prototyping-Modell; IE11 unterstützt das Schlüsselwort class nicht. Klassen sind übrigens Funktionen

Syntax

Funktionen sind First-Class Objects

```
function myFunction() {  
    console.log("Hello, world!");  
}  
  
myFunction.someProperty = "foo";  
  
console.log(myFunction.someProperty); // Output: "foo"  
  
let f = function myFunction() {  
    console.log("Hello, world!");  
}  
  
f.someProperty = "foo";  
  
console.log(f.someProperty); // Output: "foo"
```

Syntax

Hoisting – Deklaration wird vorgeschoben, Initialisierung nicht

In JavaScript werden Variablen- und Funktionsdeklarationen an den Anfang des Gültigkeitsbereichs verschoben

Daher kann man eine **Funktionsdeklaration** im Code oft schon **vor ihrer Definition** aufrufen. Die Initialisierungen und Zuweisungen von Variablen und Konstanten finden allerdings an der Stelle statt, an der sie stehen.

Best Practices zur Vermeidung von Problemen

1. Variablen immer vor ihrer Verwendung deklarieren
2. Nutze wenn möglich const, denn hier wird gleichzeitig initialisiert – Kein Hoisting Problem
3. Aufpassen, wenn const eine Funktion ist (ggf. Reference Error)
4. Vermeide komplexe Verschachtelungen von Scopes

Syntax

Vorsicht bei der erlaubten frühzeitigen Verwendung

```
console.log(x); // ReferenceError:
                // Cannot access 'x' before initialization
let x = 10;
console.log(x); // Ausgabe: 10
```

Die Initialisierung von `let` und `const` Werten findet in der **Temporal Dead Zone (TDZ)** statt. Die Variablen dürfen vorher **nicht gelesen/benutzt** werden → sonst `ReferenceError`

Syntax

Vorsicht bei der erlaubten frühzeitigen Verwendung

```
let carName = "Volvo";  
myFunction();
```

```
function myFunction() {  
  
    console.log(carName);  
}  
myFunction();
```

Volvo
Volvo

```
myFunction();  
let carName = "Volvo";  
myFunction();
```

```
function myFunction() {  
  
    console.log(carName);  
}  
myFunction();
```

ReferenceError: Cannot access 'carName' before initialization

Syntax

Der einfache Weg hin zu Objekten

Instanziierung eines Objektes

```
var user = new Object(); // Nutzung des Keywords "new"  
var user = {};           // JSON-like Schreibweise
```

> Identisches Ergebnis, unterschiedliche Schreibweise

Anschließende Zuweisung der Attribute (jederzeit nach Instanziierung)

```
user.id = 12;                // Attribut id hinzufügen  
user.name = 'Max Mustermann'; // Attribut name hinzufügen  
user['name'] = 'Max Mustermann'; // Alternative - wie Arrays
```

Alternative (nur für die JSON-like Schreibweise)

- Attribute sofort bei der Instanziierung setzen

```
var user = {  
  id : 12,  
  name : "Max Mustermann"  
};
```

Auslesen über .-Operator oder eckige Klammern

- Analog zur Zuweisung
- Beispiele:

```
var id1 = user.id;    // 12  
var id2 = user['id']; // Alternative wie Arrays
```

Als Wert eines Attributes ist alles möglich, die Bezeichnung ist ein String

Syntax

Funktionen, Gültigkeitsbereich und Variablen

Beispiel:

```
function fak(n) {  
    if (n <= 1) {  
        return 1;  
    }  
    return n * fak(n - 1);  
}
```

- Erstellt eine Funktion und legt diese unter dem Namen „fak“ im aktuellen Geltungsbereich der Variablen (Scope) ab. Tatsächlich kann die Funktion auch schon vor ihrer Deklaration verwendet werden (Hoisting).
- Wenn innerhalb von Funktionen Variablen ohne das Schlüsselwort `var` oder `let/const` verwendet werden, dann sind diese Variablen global, wurden also extern bereits erstellt
 - > Für lokale Variablen sicherstellen, dass `var`, `let` oder `const` verwendet wurde
 - > **Parameter sind immer lokal**

Beispiel (als anonyme Funktion):

```
let fak = function (n) {  
    if (n <= 1) {  
        return 1;  
    }  
    return n * fak(n - 1);  
};
```

- Erstellt eine anonyme Funktion und **speichert den Funktionszeiger** in der Variable „**fak**“
- „fak“ wird im aktuellen Scope abgelegt (hier durch var ein Function-Scope)
- Äquivalent zum ersten Ausdruck
 - > **Fast**, denn die Funktion wird hier erst bekannt, wenn die Zuweisung let fak = erreicht wird. Bei benannten Funktionen sind diese auch schon vorher bekannt und können schon **vor Deklaration** genutzt werden (Hoisting vs. Zuweisung)

Syntax

Funktionen, Gültigkeitsbereich und Variablen

Anonyme Funktionen

- Schnelle Erzeugung ohne Speicherung eines Bezeichners im Geltungsbereich
- Häufig verwendet bei Eventhandlern, die direkt fest zugeordnet werden:

```
einHTMLelement.onclick = function () {  
    alert('Geklickt!');  
}
```

- **Typisch für JavaScript:** Kapselung des Skriptes um Namenskonflikte zu vermeiden. Wir verpacken also unsere verwendeten Variable in einer Funktion (**sicherer neuer Scope**), die dann auch direkt aufgerufen wird

```
(function() {  
    let info = 'Hallo'; // Nur innerhalb sichtbar  
    alert(info);  
})();  
// Variable info ist in diesem Geltungsbereich unbekannt  
// Es gibt auch keinen etwaigen Konflikt mit dem Namen
```

Syntax

Funktionen, Gültigkeitsbereich und Variablen

Vergleich zu anderen Sprachen:

- C++-Code (!!!):

```
int x = 1;
if (true) {    // neuer Geltungsbereich!
    int x = 2; // neue "innere" Variable x
}
// x hat den Wert 1
```

Syntax

Funktionen eröffnen einen neuen Geltungsbereich

JavaScript-Code:

```
var x = 0;
(function() {           // Neuer Geltungsbereich
    var x = 1;
    if (true) {
        var x = 2; // JavaScript ignoriert das "var", da
                    // es bereits eine Variable x
                    // im aktuellen Scope gibt. Das wird
    }               // bei let anders sein
    // x hat den Wert 2
})();
// x hat den Wert 0
```

Funktionen erstellen immer einen neuen Scope

- Variablen am Anfang einer Funktion deklarieren
- Bei **let** erstellen auch geschweifte Klammern einen neuen Scope

JavaScript-Code mit let (seit ECMAScript 6 2015):

```
var x = 0;

(function() {      // Neuer Geltungsbereich
    let x = 1;
    if (true) {
        let x = 2; // Neue Variable x im inneren Bereich
    }
    // x hat den Wert 1
})();
// x hat den Wert 0
```

```
let x = 1;
let x = 2;
console.log(x);
```

let x = 2;
^
SyntaxError: Identifier 'x' has already been declared

Syntax

let vs. var

```
function abc() {  
    //i *ist* auch hier sichtbar  
    for( var i = 0; i < 5; i++ ) {  
        // i ist in der ganzen Funktion sichtbar  
    }  
    // i *ist* auch hier sichtbar  
}
```

```
function abc() {  
    //i *ist* hier nicht sichtbar  
    for( let i = 0; i < 5; i++ ) {  
        // i ist nur in diesem Block sichtbar  
    }  
    // i *ist* hier nicht sichtbar  
}
```

Syntax

const in Schleifen

```
function abc() {  
    //i *ist* hier nicht sichtbar  
    for( const i = 0; i < 5; i++ ) {  
        // i ist nur in diesem Block sichtbar  
        // tatsächlich hat jeder Schleifendurchlauf ein eigenes i  
        // damit ist dies sogar konstant  
    }  
    // i *ist* hier nicht sichtbar  
}
```

Syntax

const ist nicht „konstant“ (Objektwerte können verändert werden)

```
const SomeOb = {  
    id: 233,  
    name: null,  
};  
  
// allowed  
SomeOb.name = "John Doe";  
  
// not allowed as variable cannot be reassigned  
SomeOb = 10;
```


Vor let gab es viele Probleme durch „fehlenden“ Geltungsbereich

- Beispiel:

- > Wir assoziieren den onclick-Event mit einigen DOM-Elementen

- ```
var domelements = document.getElementsByClassName('name');
```

- > Für das erste Element soll beim Klick 0 ausgegeben, für das zweite 1, usw.

```
for (var i = 0; i < 10; i++) {
 domelements[i].onclick = function () {
 alert(i);
 };
}
```

- **Problem:** Nach dem Durchlaufen hat *i* den Wert 10

- Unabhängig davon welches Element geklickt wird, hat *i* den Wert 10

- > Bis wir hier die Maus betätigen ist die Schleife schon beendet

- > Wenn wir dann später drücken, dann haben wir die Schleife bereits verlassen, die **alert(i)-**Funktion **greift** aber **auf das aktuelle i des Geltungsbereichs zu**

# Syntax

## Funktionen als Werkzeug zum Erzeugen eines Geltungsbereichs -> Closure

### Mögliche Lösung: Kapseln in einer Funktion mit Parameter (der lokal ist) (Übergang zum Closure-Ansatz)

```
for (var i = 0; i < 10; i++) {
 (function(innerI) { // innerI "hält" nun i
 domelement[innerI].onclick = function () {
 alert(innerI);
 };
 })(i);
}
```

- **Neuer Geltungsbereich** mit Variable *innerI*
- *Damit haben wir durch den Aufruf (i) in der Schleife jeweils den Wert gesichert*
- Faktisch wurden Closures in JavaScript auch aufgrund der unpraktischen Gültigkeitsbereiche von var in Kombination mit Hoisting so richtig populär

Weitere Informationen: <https://wiki.selfhtml.org/wiki/JavaScript/Funktion>

# Syntax

## let bietet Vorteile bei Funktionen

### Mögliche Lösung: „let“ oder const-Schlüsselwort



```
for (const i = 0; i < 10; i++) {
 domelement[i].onclick = function () {
 alert(i);
 };
}
```

- Variablen sind jetzt an Block gebunden (statt an Funktion!)
  - > Faktisch wird mit der Deklaration im For-Statement für jeden Durchlauf ein eigenes i angelegt

```
var a = 5;
{ let a = 6; console.log(a); // 6
}
console.log(a); // 5
```

Weitere Informationen: <https://developer.mozilla.org/docs/Web/JavaScript/Reference/Statements/let>, <http://caniuse.com/#feat=let>

### Variable Anzahl an Parametern möglich

- „Magische“ Variable `arguments` ist in jeder Funktion verfügbar
  - > Array-ähnliches Objekt, das die übergebenen Parameter enthält
  - > Beispiel:

```
function sumAll() {
 let sum = 0;
 for (const i=0; i<arguments.length; i++) {
 sum += arguments[i];
 }
 return sum;
}
```

### Rückgabewert kann ein beliebiger Wert (auch Objekt sein)

Weitere Informationen: <https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Functions/arguments>

# Syntax

## Funktionen – immer wieder auf den Scope achten

- Das folgende Beispiel funktioniert nicht so gut...

```
function getId() {
 let counter = 0;
 counter++;
 return counter;
}
```

```
let id1 = getId() // 1
let id2 = getId(); // 1
let id3 = getId(); // 1
```

Wir brauchen so etwas wie statische Variablen

Seit ES6 gibt es das Schlüsselwort „class“, hier gibt es die Möglichkeit Attribute mit static auszuzeichnen. Faktisch wird hier Object.defineProperty() während der Klasseninitialisierung gemacht. Diese findet bei der Deklaration statt, allerdings kein Hoisting!

### Closures

- Wir verpacken die Funktion zum Zählen in eine anonyme Funktion, die den Geltungsbereich für die Zählvariable erstellt

```
function idGetter() {
 let counter = 0;
 return function() {
 counter++;
 return counter;
 };
};

let getId = idGetter();
let id1 = getId(); // 1, setzt counter (in idGetter) auf 1
let id2 = getId(); // 2
let id3 = getId(); // 3
```

- Direkter Zugriff auf die Variable *counter* ist nicht möglich! Wir sprechen vom Lexical Scope (Zugriff auf Parent Scope möglich)

### Closures

- Vereinfachte Schreibweise

```
let getId = (function () {
 let counter = 0;
 return function() {
 counter++;
 return counter;
 };
})();
```

```
let id1 = getId(); // 1
let id2 = getId(); // 2
let id3 = getId(); // 3
```

## JSON: JavaScript Object Notation

- JSON-Dokumente sind gültiges JavaScript
- Einfache Struktur
- Typisierung eingebaut!
- Beispiel:

```
{
 "id" : 2648,
 "Name" : "Mustermann",
 "Vorname" : "Max",
 "adr" : {
 "Stadt" : "Aachen",
 "plz" : 52064
 },
 "tel" : ["0241 1234", "0160123456"],
 "partner" : null,
 "maennlich" : true
}
```

Weitere Informationen: <http://www.json.org/>



## Typen:

- Number (wie in JavaScript)
- String (nur in doppelten Anführungszeichen)
- Boolean (true/false)
- Array (nur in eckigen Klammern)
- Object (in geschweiften Klammern)
- Null

## Deckt nicht alle möglichen JavaScript-Werte ab

- NaN, Infinity werden zu null serialisiert
- Function- und RegExp-Objekte werden verworfen

### Variable Verarbeitung eines JSON-Objekts:

```
let keys = Object.keys(o);
let values = Object.values(o);
for (j=0; j < keys.length; j++) {
 console.log(keys[j] + " " + values[j]);
}
```

#### Kürzer:

```
Object.keys(obj).forEach(e => console.log(`key=${e} value=${obj[e]}`));
```

Hier würden dann aber auch etwaige mit dem Objekt verknüpfte Funktionen als Key ausgegeben und als Value der Funktionstext

### Variable Verarbeitung eines JSON-Objekts:

```
function DisplayObjectProperties(obj) {
 var text = "";
 for (prop in obj) {
 if(typeof obj[prop] != "function" &&
 typeof obj[prop] != "object") {
 text = text + prop + ": " + obj[prop] + " " ;
 }
 else if (typeof obj[prop] === "object") {
 DisplayObjectProperties(obj[prop])
 }
 }
 console.log(text) ;
}
DisplayObjectProperties(einObjekt) ;
```

# Syntax

## Objekte – Wir können hier auch Methoden definieren

### Unser erstes „richtiges“ Objekt

```
var auto = {
 maxSpeed : 140,
 name : 'Corsa',
 distance : 0,
 go : function(times) {
 auto.distance += auto.maxSpeed * times;
 },
 getDistance : function() {
 return auto.distance;
 }
};

auto.go(2);
alert(auto.getDistance()); // 280
```

### Unser erstes „richtiges“ Objekt (verbessert)

```
var auto = {
 maxSpeed : 140,
 name : 'Corsa',
 distance : 0,
 go : function(times) {
 this.distance += this.maxSpeed * times; // this
 },
 getDistance : function() {
 return this.distance; // this
 }
};

auto.go(2);
alert(auto.getDistance()); // 280
```

# Syntax

## die Besonderheit - this

### Die Verwendung des this-Objekts ist in JavaScript sehr ungewöhnlich

- Innerhalb jeder Funktion verfügbar
- Es gibt jedoch Funktionszeiger und anonyme Funktionen
- **Bei Verwendung von Funktionszeigern und anonymen Funktionen (z.B. bei Event-Callbacks) zeigt this auf das Objekt, von wo aus der Aufruf statt findet**
- Der this-Wert wird **nicht durch den Ort der Definition**, sondern **durch die Art des Funktionsaufrufs** bestimmt
- Beispiel:

```
document.getElementById('btn').onclick = function() {
 this.innerHTML = 'GEKLICKT!'; // this enthält btn-Objekt
};
```

- Wurde eine solche Funktion von keinem Objekt aufgerufen, so enthält this im Browser das globale Objekt window
  - > window ist das „globale“ Browserobjekt

Mehr zu window: <https://developer.mozilla.org/docs/Web/API/Window/window>

### „Herausziehen“ von Funktionen kann zu Probleme führen

- Beispiel:

```
document.getElementById('btn').onclick = auto.go;
```

```
// ...
var auto = {
 // ...
 go : function () {
 // this zeigt auf den btn-Objekt Button
 },
 // ...
};
```



Nur ein Funktionszeiger.  
Der Aufruf wird daher über das btn-  
Element im Falle eines Events über den  
onclick-Funktionszeiger durchgeführt

### Sauberes kapseln wäre hier zwingend erforderlich

- Beispiel:

```
document.getElementById('btn').onclick = function () {
 auto.go();
};

// ...
var auto = {
 // ...
 go : function () {
 // this zeigt auf auto
 },
 // ...
};
```



Auch ein Funktionszeiger.  
Der Aufruf wird aber hier im Innern  
der anonymen Funktion durch das  
auto-Objekt ausgeführt



### Manipulation des this-Zeigers, Beispiel:

```
var auto = { // ...
 go : function (times) {
 this.distance += this.maxSpeed * times;
 }, // ...
};
```

```
var auto2 = {
 maxSpeed : 100,
 distance : 0
};
auto.go.apply(auto2, [3]);
alert(auto2.distance); // 300
```

auto2 bei Ausführung der Funktion go als this-Objekt nutzen

Parameter als Array

> Wendet Funktion go des Objektes auto auf das Objekt auto2 an.

Weitere Informationen: [https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Global\\_Objects/Function/apply](https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Global_Objects/Function/apply)

### Manipulation des this-Zeigers

```
Function.apply(thisArg [, argsArray])
```

- Führt die Funktion aus und setzt den this-Zeiger auf thisArg
- Werte aus argsArray werden der Funktion als Parameter übergeben
  - > Sinnvoll, wenn unklar ist, wie viele Parameter übergeben werden

```
Function.call(thisArg [, arg1 [, arg2 [, ...]]])
```

- Führt die Funktion aus und setzt den this-Zeiger auf thisArg
- Parameter werden als normale Parameter übergeben

### Wir können auch den This-Zeiger bewahren

```
function myFunction(a, b, c) {
 console.log(this.x + a + b + c);
}

const myObject = { x: 10 };
const boundFunction = myFunction.bind(myObject, 1, 2, 3);
boundFunction(); // Output: 16
```

bei Function.bind geben wir das Objekt an, das wir dann später über den this-Zeiger verwenden.

**Im Gegensatz zu apply gibt es hier keinen Aufruf**

### Konstruktorfunktion

- Funktionen sind First-Class-Objects! Jede Funktion kann somit auch als Konstruktor genutzt werden
- Beispiel:

```
// Diese Funktion wollen wir als Konstruktor nutzen
function Fahrzeug(speed) {
 this.speed = speed;
 this.distance = 0;
};
```

```
// Erstellen eines neuen Objektes des Typs Fahrzeug
var fahrzeug1 = new Fahrzeug(120);
```

```
alert(fahrzeug1.speed); // 120
```

- **Achtung: Man kann Fahrzeug auch ohne new aufrufen.**  
This zeigt in diesem Fall auf das window-Objekt

# Syntax

## Objekte – Self-Invoking-Konstruktor

### Konstruktorfunktion

- Um das this-Problem der Nutzung der Konstruktorfunktion ohne vorheriges new zu unterbinden verwendet man gerne einen Self-Invoking-Konstruktor
- Beispiel:

```
function Fahrzeug(speed) {
 if (!(this instanceof Fahrzeug)) {
 return new Fahrzeug(speed);
 }
 this.speed = speed;
 this.distance = 0;
};

new Fahrzeug(120);
Fahrzeug(120); // Funktioniert auch
```

- Seit ECMAScript 2015 können wir Objekte auch syntaktisch als Klasse darstellen (**kein echtes OOP-Modell, syntaktischer Zucker**)

```
class Rechteck {
```

```
 #breite = 0;
```

```
 constructor(hoehe, breite) {
```

```
 this.#hoehe = hoehe;
```

```
 this.#breite = breite;
```

```
 }
```

```
 flaeche() {
```

```
 return this.#hoehe * this.#breite;
```

```
 }
```

```
}
```

Wichtig: Private Variablen müssen vor Verwendung deklariert werden. Eine Zuweisung ist hier nicht notwendig

SyntaxError: Private field '#hoehe' must be declared in an enclosing class

- Die #-Notation ist seit ECMAScript 2022 der offizielle Weg, **private** Variablen zu deklarieren.
- Aufpassen: const kann nicht mit privaten Feldern kombiniert werden**

# Syntax

## Hoisting und Inline-Definition

---

- **Im Gegensatz zu normalen Funktionsdeklarationen** kann eine Klasse erst dann verwendet werden, wenn sie vorher deklariert wurde
- Dies gilt im übrigen auch für die statischen Attribute!!!!
- Beispiel:

```
let k = new EineKlasse(); // Ergibt einen Reference-Fehler
```

```
class EineKlasse{...};
```

- Eine Klasse kann auch anonym und direkt einer Variablen zugewiesen werden
- Beispiel

```
let var = class {...};
```

```
let var = class EineKlasse{...};
```

- Wir können wie gewohnt Methoden in einer Klasse definieren (ohne `function`)
- Beispiel:

```
class Rechteck {
 constructor (hoehe, breite) {
 this.#hoehe = hoehe;
 this.#breite = breite;
 }
 get flaeche() { //getter mit schlüsselwort get
 return this.berechneFlaeche();
 }
 berechneFlaeche() { // klassische methode
 return this.#hoehe * this.#breite;
 }
}

const r = new Rechteck(5,10);
console.log(r.flaeche); // getter, über Attribute zugreifen
```



# Syntax

## Klassen – Private-Variablen

```
class Rechteck {
 #hoehe; // Privates Instanzattribut
 #breite; // Privates Instanzattribut
 #faktor = 2.54*2.54; // Const ist verboten, aber ohne Getter oder Setter ist das quasi const

 constructor(hoehe, breite) {
 this.#hoehe = hoehe;
 this.#breite = breite;
 }

 // Getter für Höhe
 get hoehe() {
 return this.#hoehe;
 }

 // Setter für Höhe
 set hoehe(hoehe) {
 this.#hoehe = hoehe;
 }

 flaeche(inSquareFeet = false) {
 if (inSquareFeet)
 return this.#hoehe * this.#breite / this.#faktor;
 else
 return this.#hoehe * this.#breite;
 }
}

const r = new Rechteck(5, breite = 10);
console.log(r.flaeche()); // Ausgabe: 50
console.log(r.flaeche(true)); // Ausgabe: 7.75
console.log(r.hoehe); // Ausgabe: 5
r.hoehe = 20;
console.log(r.hoehe); // Ausgabe: 20
```

# Syntax

## Statische Methoden

- Beispiel:

```
class Rechteck {
 // ... siehe vorherige Folie
 static einAttribut = 42; // Erst nach Deklaration da!
 static compare(r1, r2){
 if (r1.flaeche() > r2.flaeche) {
 return r1;
 }
 else
 return r2;
 }
}

const r1 = new Rechteck(5,10);
const r2 = new Rechteck(10,10);
console.log(Rchteck.compare(r1,r2).flaeche);
```

- Wir können Klassen ableiten – anscheinend doch wie üblich
- Beispiel:

```
class Tier{
 #name;
 constructor(name) { this.#name = name; }
 sprich() {
 console.log(this.#name + ' macht ein Geräusch'); } }

class Hund extends Tier{
 sprich() { console.log(this.#name + ' wau'); }
}

let h = new Hund('Wuffii');
h.sprich();
```

# Syntax

## Static liefert eine Alternative zu Closures

---

### Erinnerung: Closures

- Vereinfachte Schreibweise

```
let getId = (function () {
 let counter = 0;
 return function() {
 counter++;
 return counter;
 };
})();
```

```
let id1 = getId(); // 1
let id2 = getId(); // 2
let id3 = getId(); // 3
```

# Syntax

## Alternative zu Closures durch Class

```
class Counter {
 constructor() {
 this.counter = 1;
 }
 getId() {
 return this.counter++;
 }
}
```

```
let counter = new Counter();
console.log(counter.getId()); // Output: 1
console.log(counter.getId()); // Output: 2
```

```
class Counter {
 static counter = 1;
 static getId() {
 return Counter.counter++;
 }
}

console.log(Counter.getId()); // Output: 1
console.log(Counter.getId()); // Output: 2
```

### Arrow Funktion (wichtiges Hilfsmittel)

- Gern genutzte Kurzschreibweise für Funktionen
- Zusätzlich **übernimmt** die aufgerufene Funktion den **this-Zeiger** und bindet diesen innerhalb der Funktion nicht neu - ähnlich wie bei einer vorherigen Bindung mit bind()
- Typische Einsatzfälle sind **anonyme Callbacks** (z. B. in Event- oder Promise-Code) sowie **kurze Transformationsfunktionen** („Lambdas“)

```
[1,2,3].map((value) => value + 1);
```

```
function Person() {
 this.age = 0;

 setTimeout(() => {
 this.age++; // this zeigt immer noch auf Person
 console.log('Alter: ' + this.age);
 }, 1000);
}
var person = new Person();
```

Weitere Informationen: [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow\\_functions](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions)

### Arrow Funktion (wichtiges Hilfsmittel)

- Gern genutzte Kurzschreibweise für Funktionen
- Zusätzlich **übernimmt** die aufgerufene Funktion den **this-Zeiger** und bindet diesen innerhalb der Funktion nicht neu - ähnlich wie bei einer vorherigen Bindung mit bind()
- Typische Einsatzfälle sind **anonyme Callbacks** (z. B. in Event- oder Promise-Code) sowie **kurze Transformationsfunktionen** („Lambdas“)

# Arrow-Funktion

## ECMAScript 6 – Einfach Standardfälle

```
// Kurzschreibweise (ein Ausdruck -> implizites return)
const square = x => x * x;
console.log(square(5)); // 25

// Mehrere Parameter + Block (mit explizitem return)
const sum = (a, b) => {
 const result = a + b;
 return result;
};
console.log(sum(2, 3)); // 5

// Typischer Einsatz: Transformationen (Lambdas) mit map/filter/reduce
const nums = [1, 2, 3, 4, 5];

const squares = nums.map(n => n * n); // [1,4,9,16,25]
const evens = nums.filter(n => n % 2 === 0); // [2,4]
const total = nums.reduce((acc, n) => acc + n, 0); // 15

// Objekte zurückgeben: Klammern nötig, sonst wird es als Block interpretiert
const makeUser = (name, age) => ({ name, age });
console.log(makeUser("Mina", 21)); // { name: "Mina", age: 21 }
```



# Arrow-Funktion (Wichtig!!!)

## ECMAScript 6

### Arrow Funktion

### Wichtiges Hilfsmittel im Zusammenhang mit this-Zeigerproblemen

```
function Person() {
 this.age = 0;

 setTimeout(handler: function () {
 console.log('Alter: ' + this.age);
 }, timeout: 1000);
}
let person = new Person();
```

```
function Person() {
 this.age = 0;

 setTimeout(() => {
 this.age++; // this zeigt immer noch auf Person
 console.log('Alter: ' + this.age);
 }, 1000);
}
var person = new Person();
```

Alter: undefined

This zeigt z.B. auf Window

# Arrow-Funktion (Wichtig!!!)

## ECMAScript 6

### Arrow Funktion

### Wichtiges Hilfsmittel im Zusammenhang mit this-Zeigerproblemen

```
function Person() {
 this.age = 0;

 setTimeout(() => {
 this.age++; // this zeigt immer noch auf Person
 console.log('Alter: ' + this.age);
 }, 1000);
}
var person = new Person();
```

```
function Person() {
 this.age = 0;

 const tick = function () {
 this.age++;
 console.log('Alter: ' + this.age);
 };

 const boundTick = tick.bind(this);
 setTimeout(boundTick, 1000);
}
```

Analoge Funktion ohne Arrow-Funktion

# Arrow-Funktion

[https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow\\_functions](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions)

```
const materials = [
 'Hydrogen',
 'Helium',
 'Lithium',
 'Beryllium'
];

console.log(materials.map(material => material.length));
// expected output: Array [8, 6, 7, 9]
```

```
// Parameterless arrow functions that are visually easier to parse
setTimeout(() => {
 console.log('I happen sooner');
 setTimeout(() => {
 // deeper code
 console.log('I happen later');
 }, 1);
}, 1);
```

|                              | Desktop |      |         |                   |       |        | Mobile          |                |                 |               |            |                  | Node.js |
|------------------------------|---------|------|---------|-------------------|-------|--------|-----------------|----------------|-----------------|---------------|------------|------------------|---------|
|                              | Chrome  | Edge | Firefox | Internet Explorer | Opera | Safari | WebView Android | Chrome Android | Firefox Android | Opera Android | iOS Safari | Samsung Internet | Node.js |
| Arrow functions              | 45      | 12   | 22 ★    | No                | 32    | 10     | 45              | 45             | 22 ★            | 32            | 10         | 5.0              | 4.0.0   |
| Trailing comma in parameters | 58      | 12   | 52      | No                | 45    | 10     | 58              | 58             | 52              | 43            | 10         | 7.0              | 8.0.0   |

Weitere Informationen: [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow\\_functions](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions)

## Mittels import gibt es eine einfache Möglichkeit externe Bibliotheken einzubinden

- Es gibt vier Möglichkeiten der Nutzung
  1. [Named import](#): `import { export1, export2 } from "module-name";`
  2. [Default import](#): `import defaultExport from "module-name";`
  3. [Namespace import](#): `import * as name from "module-name";`
  4. [Side effect import](#): `import "module-name";`

```
import { test_func } from 'test.js';
console.log(test_func());
```

- Weitere Details bei node.js

## Single-Threaded Non-Blocking

---

```
setTimeout(() => { console.log('I happen sooner');
 setTimeout(() => {
 // deeper code
 console.log('I happen later');
 }, 1);
}, 1);

console.log('I am actually the first');
```

Weitere Informationen: [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow\\_functions](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions)

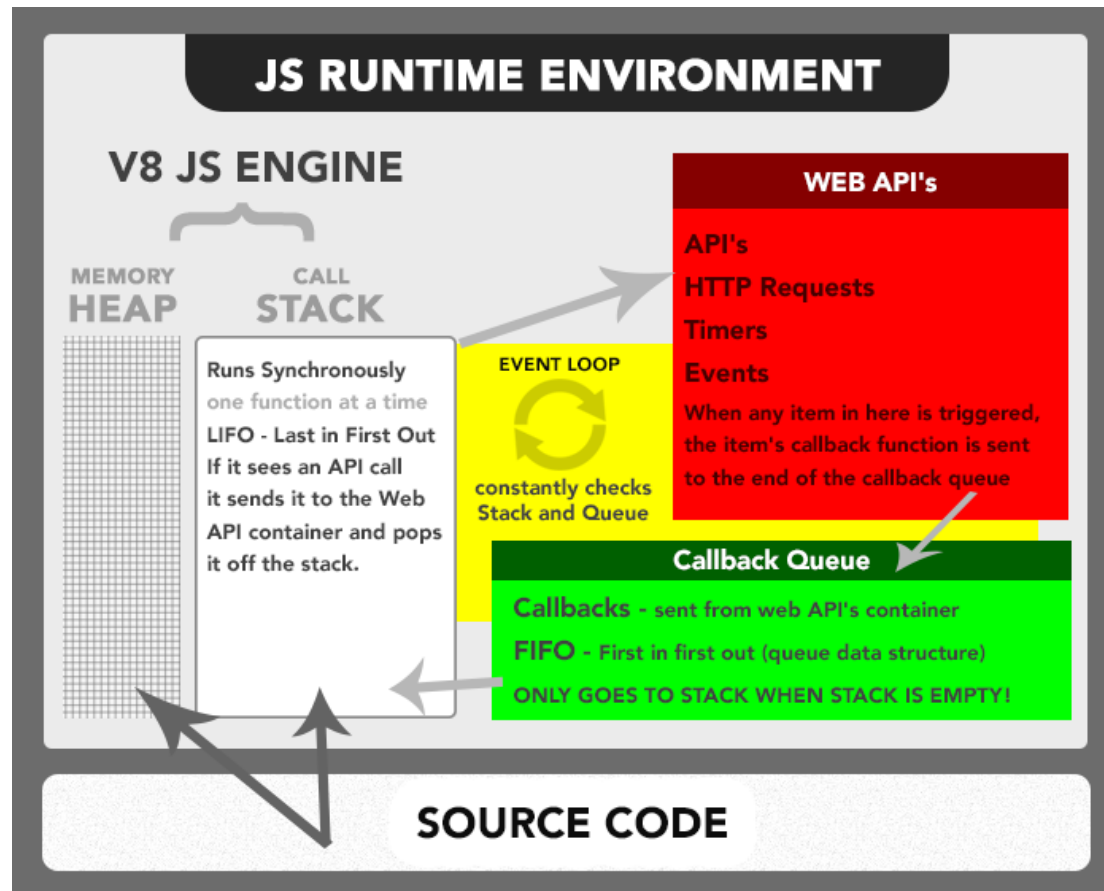
## Charakteristik einer JavaScript Runtime

„It's a single-threaded non-blocking asynchronous event-based language!“

- Ein Thread (nur einen Call-Stack)
- Nicht-blockierend (non-blocking)
  - > Techniken, die bei anderen Sprachen (z.B. Java) die Codeabarbeitung verhindern, passieren bei JavaScript im „Hintergrund“
    - Hintergrund? Ein Thread?
- Asynchron
  - > Benutzung von Callbacks um festzustellen wenn Code fertig ist
  - > Sehr passend für die Nutzung von Events

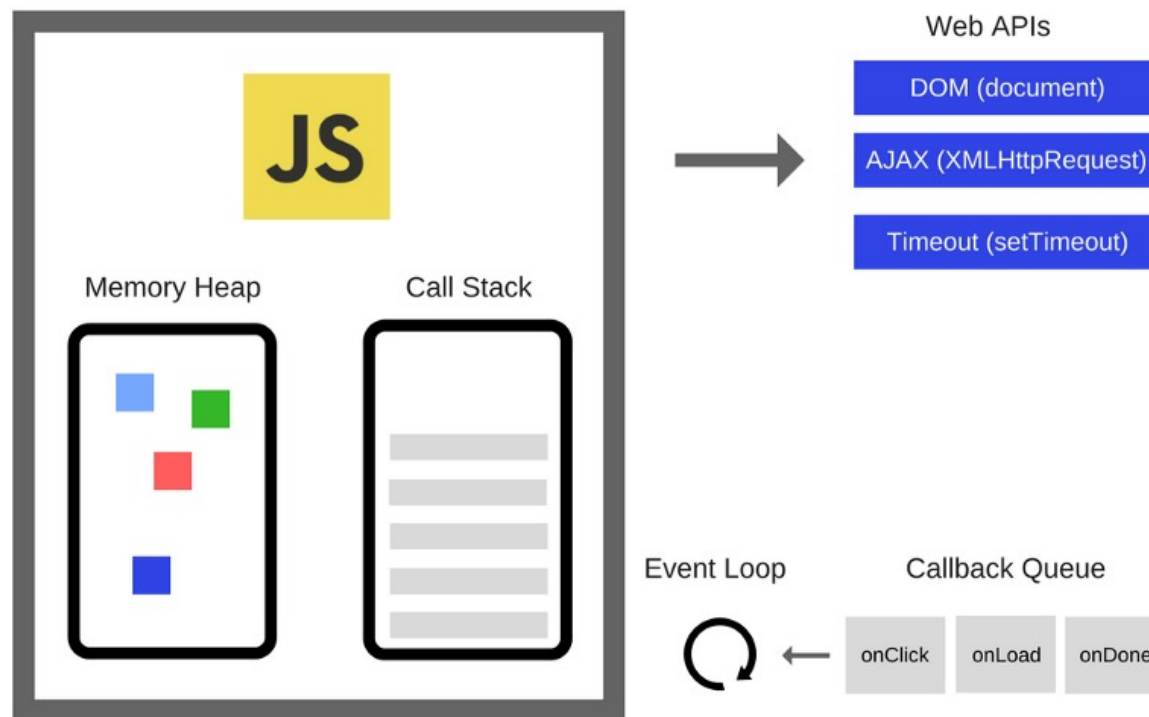
# Event Loop

<https://medium.com/@monuchaudhary/single-threaded-non-blocking-asynchronous-and-concurrent-nature-of-javascript-a0d5483bcf4c>



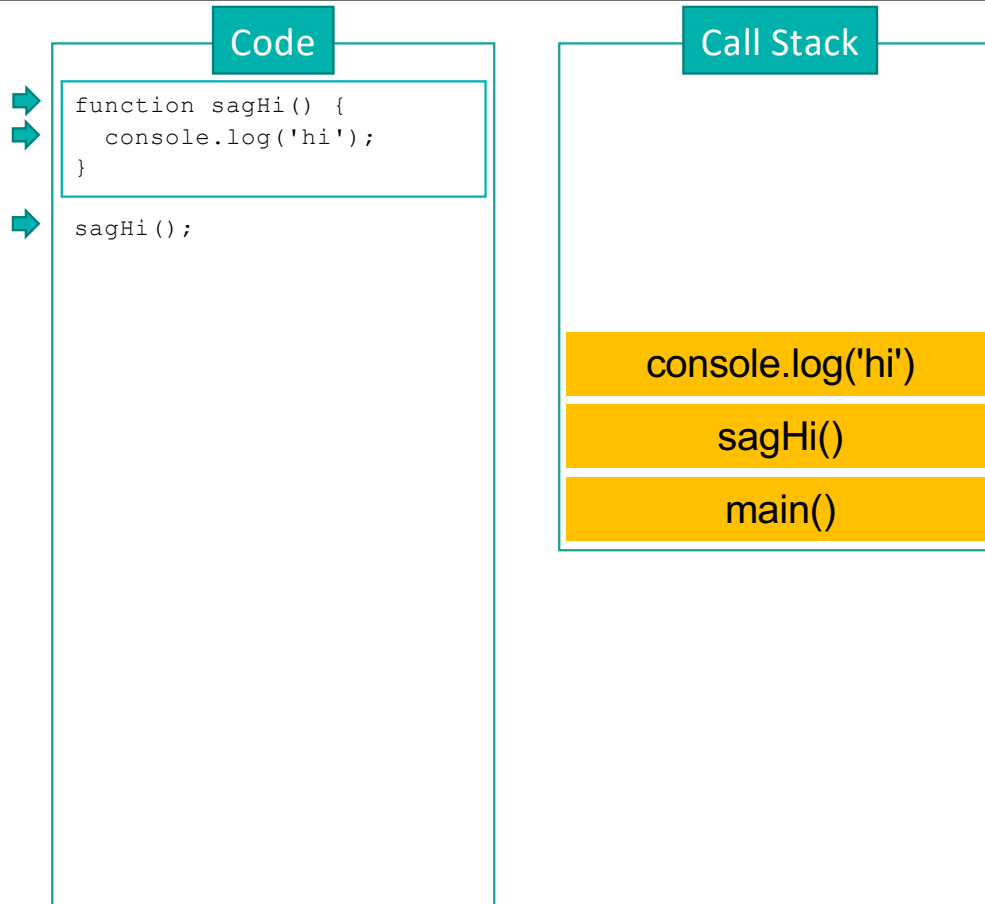
# Event Loop

<https://medium.com/@monuchaudhary/single-threaded-non-blocking-asynchronous-and-concurrent-nature-of-javascript-a0d5483bcf4c>

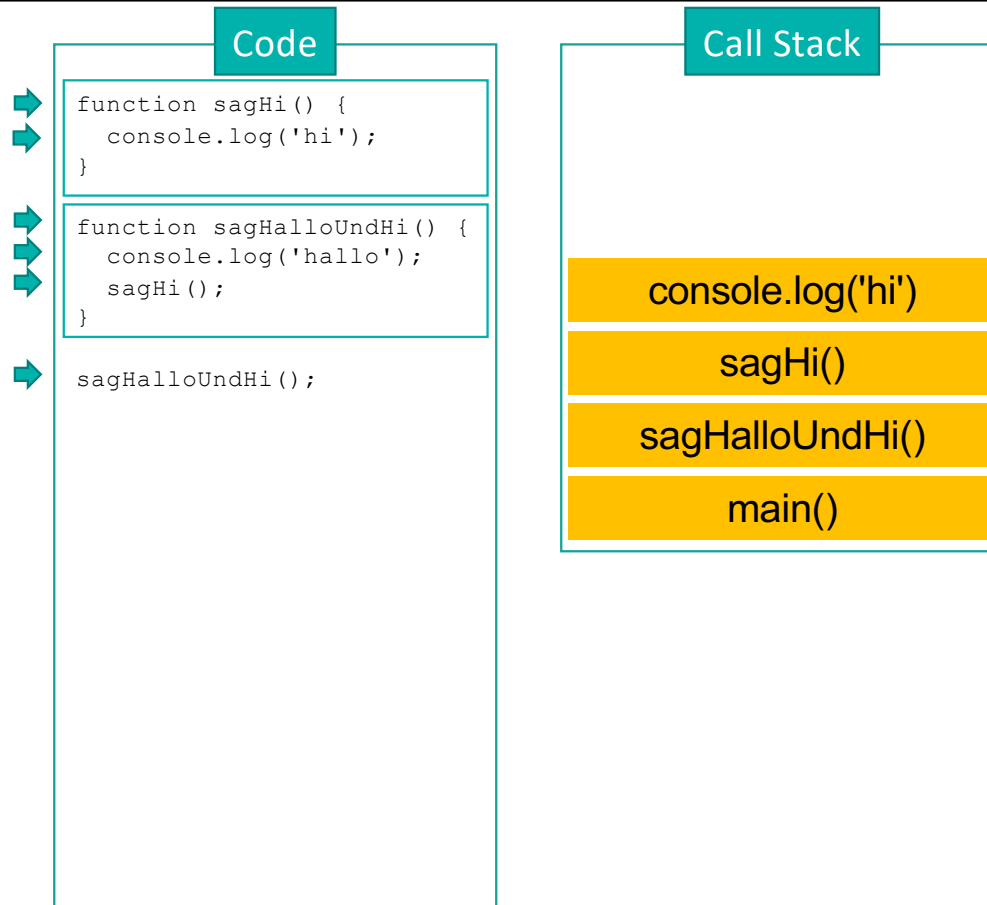




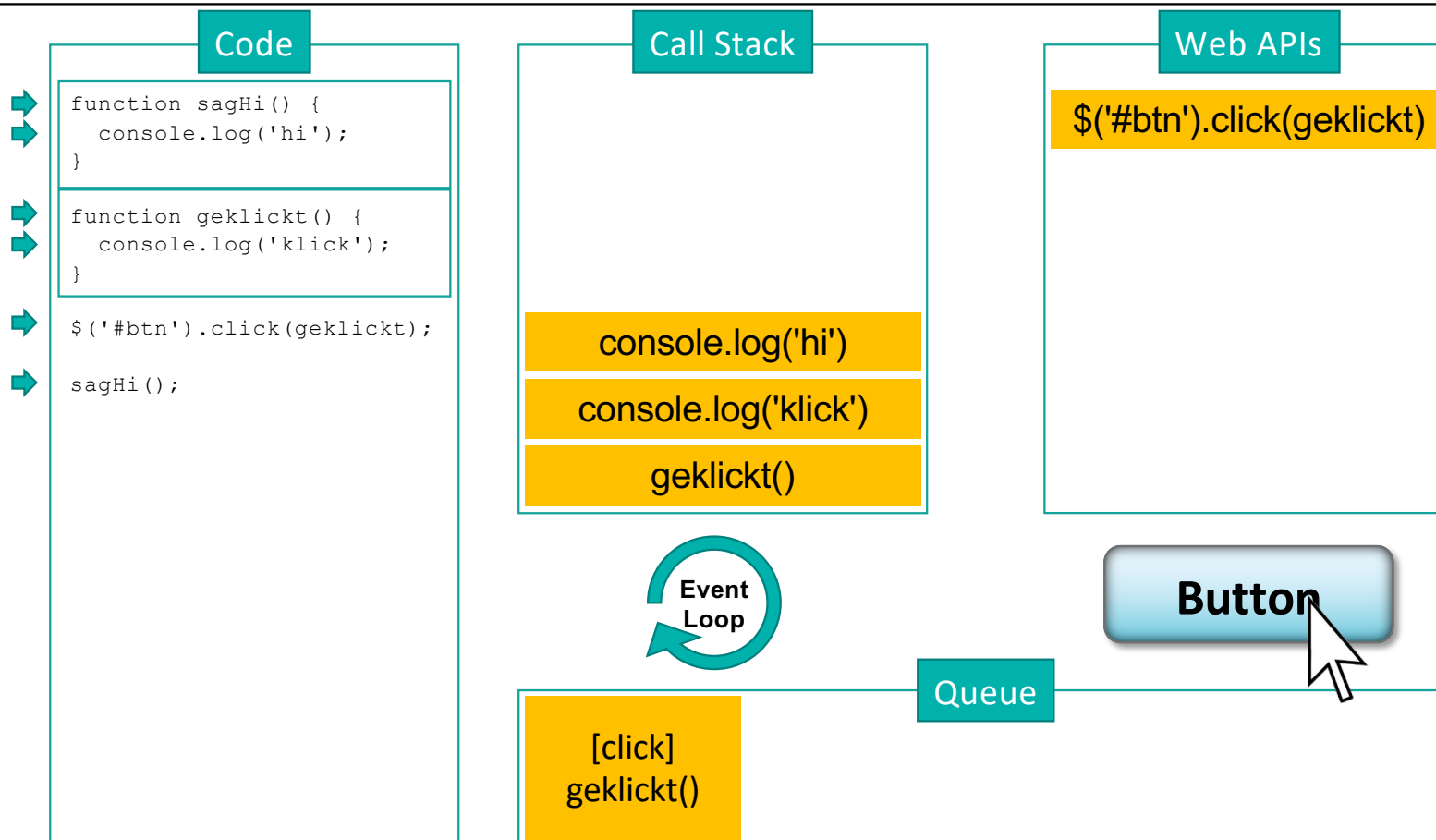
# Event Loop



# Event Loop

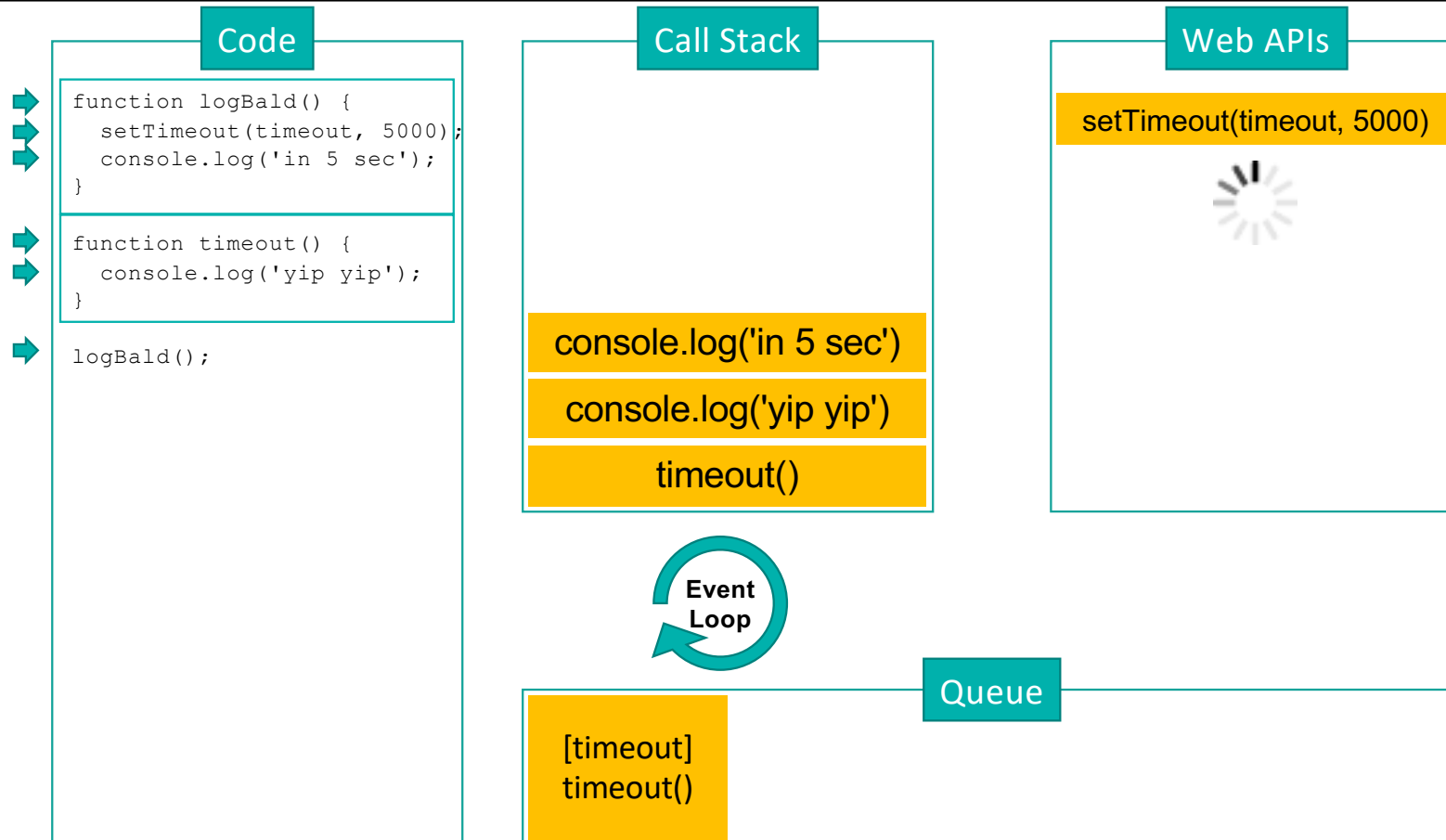


# Event Loop



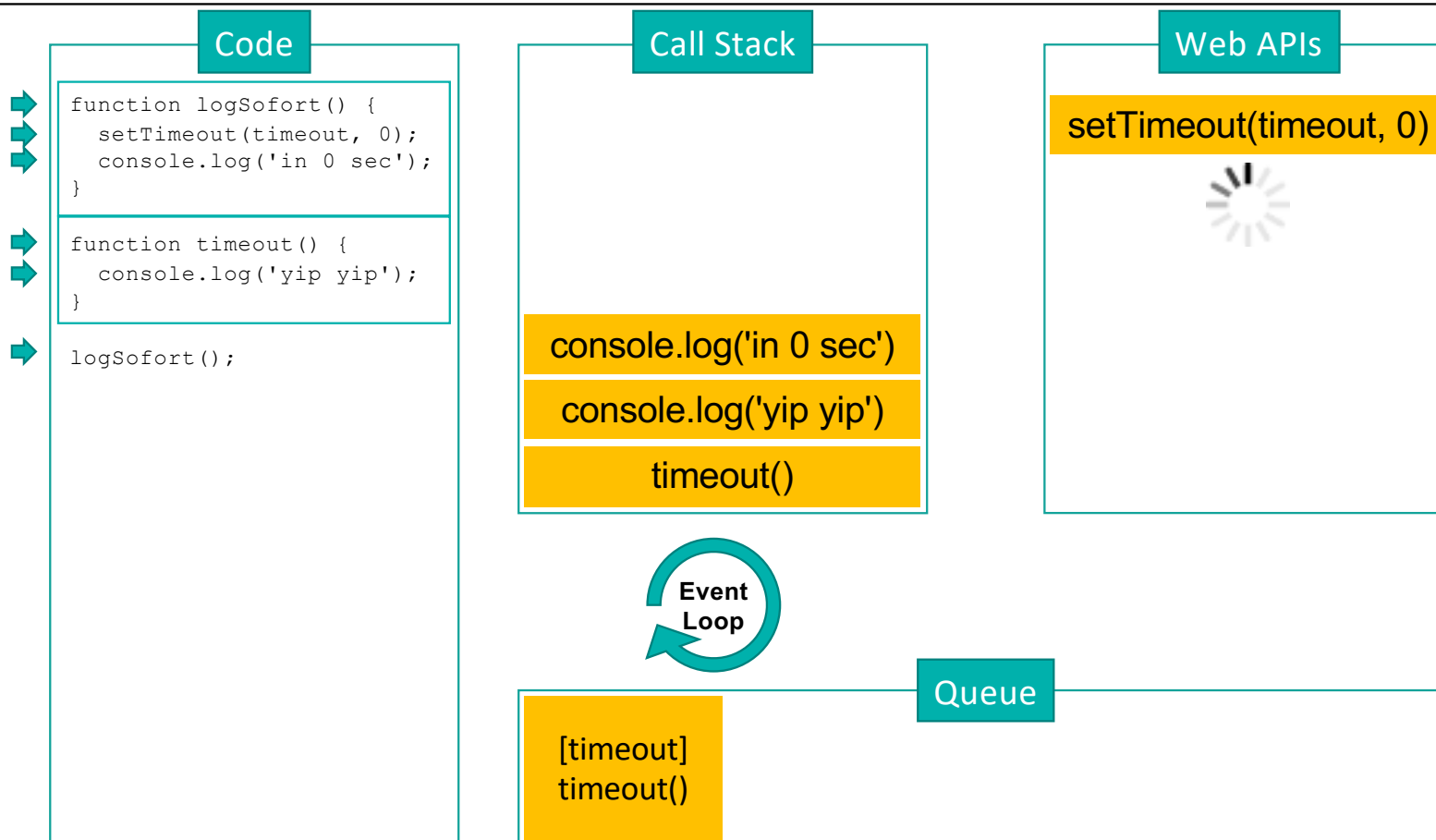
Selber ausprobieren: <http://latentflip.com/loupe/>

# Event Loop



Selber ausprobieren: <http://latentflip.com/loupe/>

# Event Loop



Selber ausprobieren: <http://latentflip.com/loupe/>

# Event Loop

---

## Funktion der Event Loop

- Programm:

```
console.log("i am first");
console.log("i am second");
console.log("i am third");
```

- Ausgabe:

```
i am first
i am second
i am third
```

# Event Loop

---

## Funktion der Event Loop

Programm:

```
console.log("i am first");

setTimeout(function timeout() {
 console.log("i am second");
}, 5000);

console.log("i am third");
```

Ausgabe:

```
i am first
i am third
i am second
```

# Event Loop

## Funktion der Event Loop

Programm:

```
console.log("i am first");

setTimeout(function timeout() {
 console.log("i am second");
}, 0);

console.log("i am third");
```

Ausgabe:

```
i am first
i am third
i am second
```

setTimeout wird zwar sofort ausgeführt, aber die 3. Konsolenausgabe ist schon vorher da



# Event Loop

## Funktion der Event Loop

- Arbeitet die Event Queue ab
  - Wenn Stack leer ist (also gerade nichts ausgeführt wird), so führe ersten Eintrag der Queue aus\*
- Erlaubt Asynchronität ohne Nebenläufigkeit
  - > Nur ein Thread für die JavaScript-Engine
  - > Implementierung des Systems um die JS-Engine herum, kann mit mehreren Threads erfolgen
  - > Probleme von parallelen Programmen werden vermieden
    - (z.B. paralleler Zugriff auf Variablen)

## Never block the Event Loop!

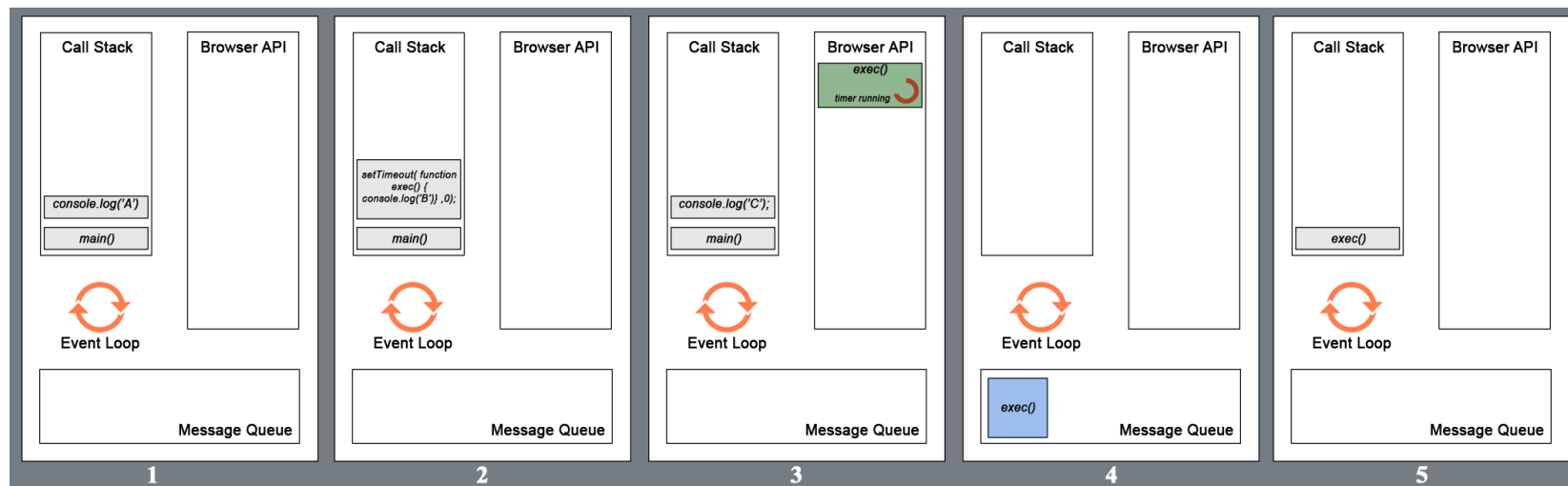
Endlosschleifen bedingen, dass keine Interaktion mit der WEB-API im Browser mehr möglich ist

\*Tatsächlich werden im Browser weitere Queues gepflegt, insbesondere die Job (Microtasks) und die Task Queue (Macrotasks)

# Event Loop

<https://medium.com/front-end-weekly/javascript-event-loop-explained-4cd26af121d4>

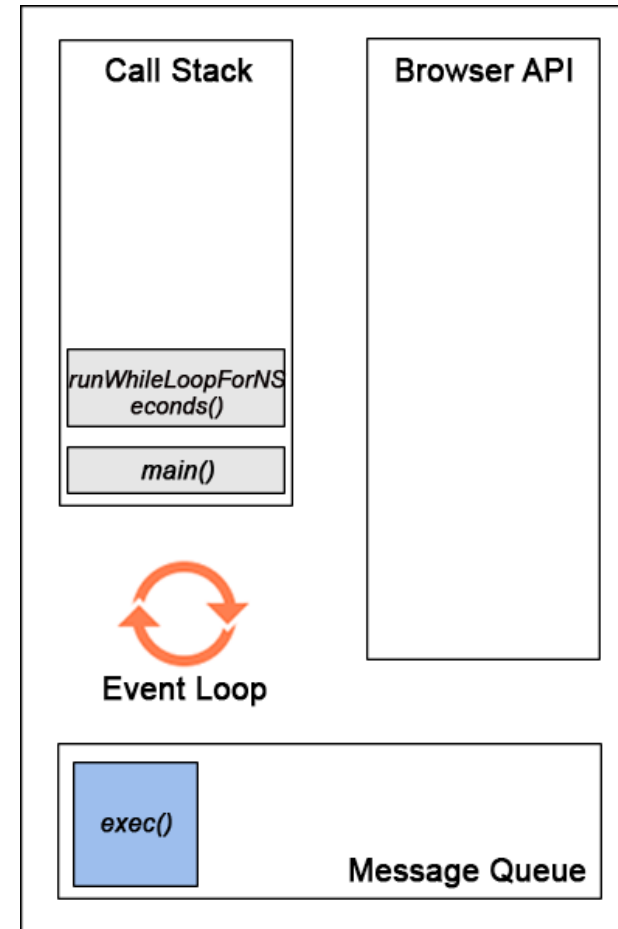
```
1 function main(){
2 console.log('A');
3 setTimeout(
4 function display(){ console.log('B'); }
5 ,0);
6 console.log('C');
7 }
8 main();
9 // Output
10 // A
11 // C
12 // B
```



# Event Loop

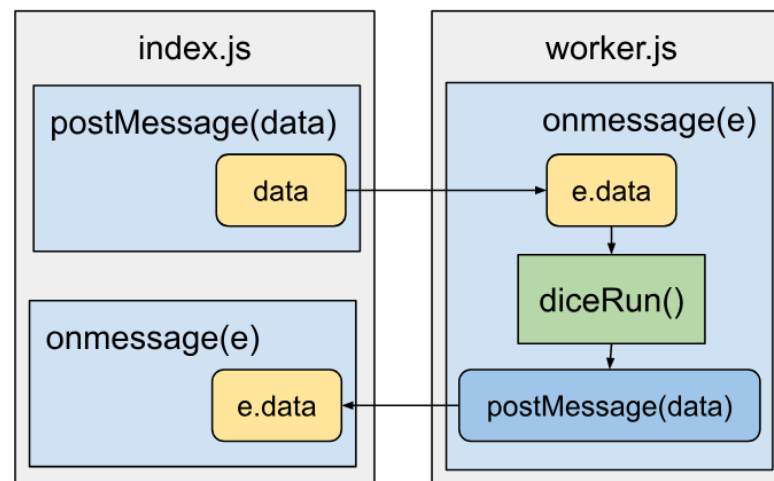
<https://medium.com/front-end-weekly/javascript-event-loop-explained-4cd26af121d4>

```
1 function main(){
2 console.log('A');
3 setTimeout(
4 function exec(){ console.log('B'); }
5 , 0);
6 runWhileLoopForNSeconds(3);
7 console.log('C');
8 }
9 main();
10 function runWhileLoopForNSeconds(sec){
11 let start = Date.now(), now = start;
12 while (now - start < (sec*1000)) {
13 now = Date.now();
14 }
15 }
16 // Output
17 // A
18 // C
19 // B
```



## Hinweis

**In den meisten modernen Browsern gibt es eine Worker-Klasse mit der sich CPU-intensive Arbeiten auslagern lassen. Wir kommen hierzu noch**



<https://www.sitepoint.com/developing-faster-javascript-apps-the-ultimate-guide-to-web-workers/>

<https://blog.pixelfreestudio.com/how-to-use-html5-web-workers-for-background-tasks/>